Agent workspace with symlinks

Share

> you could technically make an "agent first" linux distro. where the filesystem has a home for every agent running, and the filesystem access is basically the limit to what the agent can see. the human facing interface is then not so different from what composer currently is. just with extra settings for giving and taking away access to certain agents

Yes, and the core idea is sound: treat each agent as a first-class user of the OS, with the filesystem as the capability boundary. Unix already has most of the primitives. The distro would mostly be about making them default and ergonomic instead of something you bolt on.

What maps cleanly

- **One uid per agent, home dir per agent.** Plain Unix permissions give you the "filesystem is the limit" property for free. Granting access is a group membership, ACL, or bind mount; revoking is removing it.
- **Mount namespaces for the real boundary.** Permissions say what an agent *may* touch; a per-agent mount namespace (bubblewrap/Landlock style) means everything else doesn't *exist* from its point of view. That's closer to what you described: the filesystem is the agent's whole world.
- **Grants as mounts.** "Give agent X read access to ~/photos" becomes a read-only bind mount into X's namespace. It's visible, auditable, and trivially revocable.
- **Composer's UI becomes the control plane.** A matrix of agents × directories/devices/network, with a toggle per cell, is basically what you'd build anyway.

Where the filesystem alone isn't enough

- **Network.** A file boundary doesn't stop an agent from exfiltrating what it can read. You'd want per-agent network namespaces with an allowlist, ideally routed through a proxy that logs.
- **Secrets and credentials.** Agents shouldn't hold API keys directly. A broker process that injects credentials per request, scoped per agent, fits the "home per agent" model well.
- **Processes and devices.** Screen, mic, camera, input, and other agents' processes need the same treatment. This is where your wayai compositor fits naturally: the compositor decides which windows an agent can see or drive, so display access is just another grant.
- **Inter-agent communication.** Probably shared directories or sockets that are explicitly granted, so "agent A can hand files to agent B" is a visible edge in the same permission graph.

- **Prompt injection.** Permissions limit the blast radius, not the confusion. An agent with read access to email and write access to a shared folder can still be tricked into misusing both. The distro helps by making the *combination* of grants easy to see and constrain.

Why a distro rather than a layer on top

Mostly because the defaults matter. On a normal distro, "run this agent sandboxed" is opt-in and every tool does it differently. Here the sandbox would be how anything gets started. NixOS is the obvious base since declarative per-agent config (what's mounted, what's allowed) falls out of its model, and generations give you rollback of permission changes.

A reasonable path

You probably don't need to build a distro to learn whether this works. A daemon that spawns each agent in its own uid, namespaces, and home, with a grants file the Composer UI edits, is the actual product. The distro is just "ship that as the default session." Your brain embeddings server would fit as another service with its own scoped home, serving search to agents that were granted it.

What's the trust model you have in mind: agents you don't trust at all (third-party, downloaded), or your own agents where you mainly want containment from mistakes? That changes how hard the isolation has to be.

> containment from mistakes, and yeah it can be bolted onto a normal distro, just like how most distros are, its just handy to make it a distro by default if you are going to run it in ie a VM

Weighing the isolation needs against the main failure risks ▸

That simplifies things a lot. If the threat is mistakes rather than malice, you can drop most of the hard-isolation machinery (network egress controls, credential brokering, and so on) and spend the effort on making recovery cheap.

Snapshots matter more than permissions. The typical failure is an agent deleting the wrong directory or rewriting a file it shouldn't have, and a boundary only prevents that if it was drawn in the right place beforehand. Making every agent home a btrfs or ZFS subvolume, snapshotted automatically before each task or session, turns "the agent wrecked something" into a one-click rollback. For shared directories you've granted, the same snapshots apply, so even writes outside the agent's own home are undoable.

Cheap isolation is enough. Landlock or bubblewrap per agent, with grants as bind mounts, is plenty. You're protecting against an agent wandering where it shouldn't, so nothing needs to withstand an adversary. A useful default is read-only grants unless write is explicitly given, since most mistakes are writes.

Resource limits are the other half of mistakes. cgroups per agent for CPU, memory, and disk quota catch runaway loops, fork bombs, and an agent filling the disk with logs, which are more likely than anything exotic.

The VM case is the real argument for a distro. The VM is already your outer boundary, so the image can ship with the supervisor daemon, per-agent users, snapshot setup, and the compositor all preconfigured. You boot it and the agent-first layout is just how the system is. On a normal machine, running it as a layer makes more sense because you wouldn't want to commit your whole session to it. Building it as a layer first and then packaging an image (NixOS or even just a Debian image with a provisioning script) gets you both without maintaining a distro.

One design question worth settling early is whether grants are per-agent or per-task. Per-agent is simpler, but per-task ("this run can see ~/projects/foo, nothing else") matches how mistakes actually happen, since an agent usually needs wide access in general but narrow access for any particular job.

> it would of course not be the default composer project, but a layer on top of it.

Right, which makes it a clean separation: Composer stays the agent harness and UI, and this becomes an optional "sandbox backend" it can launch agents through. If Composer talks to agents through some spawn interface, the layer is just a different implementation of that interface. The default one runs the agent as a plain process under your user. The sandboxed one runs it under its own uid and namespace, with its home and grants.

That keeps Composer ignorant of the isolation details. All it needs to know is the grants model: a list of agents, each with a list of paths and a mode (read or write), plus whatever per-task overrides you add. The settings screen you described then edits that list, and the backend reads it when spawning. If the grants live in a plain file or small API rather than inside Composer's own state, the layer can be developed and tested without touching Composer at all.

Two things are worth keeping in mind so the layer stays optional rather than creeping into the core. First, the agent's view of the filesystem changes (its home is somewhere else, paths it was told about may not exist), so Composer's micro apps and anything that passes paths around should treat paths as relative to the agent's world, not absolute on the host. Second, anything Composer itself serves to the agent, like the micro app interfaces over MCP, should go through a socket or port that the backend explicitly exposes into the namespace, since that's the one hole you always need.

Since this is a layer, the first useful version is small: a spawn wrapper that takes an agent name and a grants list and starts the process in a bubblewrap namespace with a per-agent home. Everything else (snapshots, cgroups, the VM image) can be added later without changing that interface.

> yeah an agent would be a complete world. so

```
video-creator/
  index.md
  workflows/
  scripts/
  projects/
    <simlinks>
```

Checking whether symlinks outside the namespace would dangle ▸

That layout does a lot of work. The home directory is the agent: `index.md` is what it reads on start and what Composer can show in the UI, `workflows/` and `scripts/` are what it has learned to do, and `projects/` is the window onto everything else. Copying the directory copies the agent, and snapshotting it snapshots its whole state.

The symlinks in `projects/` are the interesting part, because they could be the grants themselves instead of a representation of them. Granting access is `ln -s ~/videos/foo projects/foo`, revoking is `rm`, and the settings UI is just a friendly editor over that directory. The supervisor reads `projects/` at spawn time and turns each link into a bind mount in the agent's namespace. You get no separate grants file to keep in sync, and you can inspect an agent's access with `ls -l`.

Two catches:

- **Symlinks can't express read-only vs write.** A bind mount can, so you need the mode encoded somewhere. The simplest options are `projects/ro/` and `projects/rw/` subdirectories, or a suffix convention. Subdirectories are more visible and need no parsing.
- **The agent must not be able to edit its own grants.** If `projects/` is writable by the agent and the supervisor trusts what's in it, the agent can grant itself anything by creating a link. For a mistakes-only threat model that's low risk, but it's also the kind of thing that could happen by accident. Keep `projects/` owned by you and mounted read-only into the agent's view, with the agent's own writable space being `workflows/`, `scripts/`, and probably a separate `scratch/` or `output/`.

Inside the namespace, the agent would see `projects/foo` as a normal directory (the bind mount), so it never deals with the symlink at all. It just sees a world where those folders exist.

> i would say, the simlinks are there mostly so that the agents have shared state to something. what could also just work is worktrees, that the agent gets access to for a task. but then it would have world state that could dissapear

Comparing shared-state links with per-task worktrees' lifespan risks ▸

Those are two different kinds of grant, and the layout should probably say so instead of putting both under `projects/`.

Shared state (symlinks): a long-lived directory that several agents see live, like a media library, a notes folder, or an inbox another agent drops files into. It's persistent by nature, so it never disappears, but concurrent writes can collide and there's no rollback except snapshots.

Task-scoped state (worktrees): a private checkout the agent works in and the human reviews. Mistakes are contained, because discarding the worktree is the rollback, and the diff is the natural thing for Composer's UI to show for approve or discard.

The disappearing-world problem is real, but it's mostly about what the agent expects to find. A few things make it manageable:

- **The agent's home holds only durable knowledge.** `workflows/`, `scripts/`, and `index.md` should refer to projects by name and role, never by assuming a path exists. "Check `projects/<name>` if present" is the contract.
- **Make absence a normal state.** If `projects/` is empty between tasks, the agent wakes up in a world with no active project, which is legible. A dangling symlink is worse because it looks like a bug.
- **Don't lose work at teardown.** Auto-commit to a task branch when a task ends. The checkout can vanish, but the commits live in the repo. The only thing lost is uncommitted junk. Better still, keep the worktree until the human merges or discards it, so teardown is a decision rather than a side effect.
- **Anything the agent should remember goes in its own home.** Learnings from the task get written to `index.md` or `workflows/`, not left in the worktree.

The catch is that worktrees only fit git-friendly content. Your video-creator example is the counterexample: large binary assets don't belong in git. For those, the equivalent is a copy-on-write clone (`cp --reflink` on btrfs or XFS), which is instant and free until files diverge, and then you diff or merge by hand or by tool. So the underlying primitive is "isolated copy of this tree for one task," and git worktrees are the implementation for code and text projects, reflink clones for media.

That gives the agent a world like:

```
video-creator/
    index.md
    workflows/
    scripts/
    shared/      # live symlinks, persistent, multi-agent
    task/        # ephemeral checkout for the current job, may be empty
```

Is the shared side mostly for agents handing files to each other, or also for reference material they all read?

> i mean, if we could make a fancy "worktree" like system. with symlinks then that would be cool too

Comparing symlink views with overlay filesystem semantics ▸

Plain symlinks can't do it alone, because a write through a symlink lands on the original file. The thing you're describing, a view that reads through to the real tree but keeps writes private, is a union mount, and Linux has one built in: **overlayfs**.

- **lower** is the real project or shared directory, mounted read-only.
- **upper** is a per-task directory that starts empty.
- **merged** is what the agent sees as `task/`.

Reads fall through to the original with no copying. Writes land in `upper`. A deleted file shows up as a whiteout marker in `upper`. This gives you worktree semantics for any content, including big binaries:

- **Diff** is just listing `upper`: new files, modified files, deletions. Composer's review UI can show that directly.
- **Merge** is copying `upper` down onto the real tree (or a subset of it, if you only approve some changes).
- **Discard** is deleting `upper`.
- **Teardown is a decision.** The overlay stays until you approve or reject it, same as I suggested for worktrees.

Overlayfs works unprivileged inside a user namespace on recent kernels, so it fits the same bubblewrap-style spawn wrapper as before.

Where the symlink idea still earns its place: curation. A worktree or overlay shows the whole tree, but a task often wants a *slice* ("these 12 clips, that music folder, nothing else"). A manifest of paths or globs per task can generate that slice as a symlink farm or a set of bind mounts under the lower layer. So the model becomes: manifest selects what's visible, overlay decides where writes go. The manifest is also a nice artifact to store with the task, since it records exactly what the agent could see.

A few gotchas:

- **Copy-up is whole-file.** Touching one byte of a 4 GB video copies all 4 GB into `upper`. On btrfs/XFS, a reflink-based clone avoids this, which is why I'd still keep that as the fallback for media-heavy projects.
- **Don't modify the lower layer while an overlay is mounted.** The behavior is undefined, so concurrent tasks on the same lower tree should either be read-only on lower or each get their own snapshot of it.
- **Symlinks pointing outside the namespace dangle.** Prefer bind mounts for the actual plumbing and use symlinks only as the human-facing convention.

So the layout stays as it was, with `task/` being an overlay mount built from a manifest instead of a checkout.

i didnt kow about overlayfs, but yeah it is basically what i described

Yes, it's the same idea, and it's the mechanism Docker uses for image layers: a read-only base, a writable layer on top, and the container sees the merge. So you'd be reusing a very well-trodden primitive instead of inventing a sync scheme.

It's worth trying by hand before designing anything around it, since it takes a minute:

```
mkdir lower upper work merged
echo hi > lower/a.txt
sudo mount -t overlay overlay -o
lowerdir=lower,upperdir=upper,workdir=work merged
echo changed > merged/a.txt
rm merged/a.txt
ls upper    # shows what changed
```

Writing and deleting through `merged` and then looking in `upper` makes the diff, whiteout, and discard behavior concrete. Rootless works too inside `unshare -Urm`, which is what the spawn wrapper would use.

Since you normally do a big ideation pass before building, the open design questions I'd park for later are: how per-task manifests are written (by you, or proposed by the agent and approved), what the merge UI looks like for binary files where there's no meaningful diff, and whether the shared directories get overlays too or are live-write only.

the site model that composer already has does keep working with this idea

Read memory ▸

That fits well, and your site model is probably the natural unit for all of this. A site is already "a workflow with its own harness and its own assumptions", which is what the agent home was: `index.md`, `workflows/`, and `scripts/` are the site's definition, and the sandbox is just what makes that boundary enforced instead of conventional.

The part that lines up best is sites sharing a project. The video site and the 3D site both pointing at one project, with the video agent running the 3D pipeline and reusing its assets, is the shared-state case from earlier. The project lives once, and each site sees it through its own grant:

```
projects/game/            # the real project, owned by you
sites/3d/shared/game  ->  read-write
sites/video/shared/game -> read-only (or overlay)
```

So "video agent reuses 3D assets" becomes a read-only grant on the 3D output directory, and the video agent can't wreck the 3D pipeline by accident. If it needs to run the 3D pipeline, that's a request to the 3D site (with its own world and write access), not the video agent reaching into it. That also answers your point that moving between custom harnesses loses assumptions: each site keeps its own harness and world, and the shared project is the only thing crossing the boundary.

Two things to settle in your ideation pass:

- **Who runs the 3D pipeline when video asks?** Either the video agent invokes it inside its own namespace with a write overlay on the 3D output, or it hands a job to the 3D site, which runs in its own world. The second is cleaner but needs a job interface between sites.
- **Where the overlay sits relative to a ticket.** Your orchestrator already starts fresh sessions per ticket, so a ticket is the natural lifetime for a `task/` overlay: created at ticket start, reviewed by your reviewer agent (the overlay diff is the thing it checks against the spec), merged or discarded when the ticket closes.

That second point is a nice fit: your coder → reviewer loop would review an overlay diff, and "reject" just means the overlay isn't merged.

> the orchestrator can do that, the default agent, can switch sites and then takes the personality of that site over

Tracing how each site's sandbox and permissions apply ▸

That removes the job interface between sites, and it simplifies the sandbox design: the world belongs to the site, not to an agent. The default agent is just a persona that gets loaded into whichever site world it enters, so there is one set of grants per site and no per-agent identity to manage.

A few consequences:

- **Switching is a respawn, not a mutation.** A running process can't swap its mount namespace, so "switch to the 3D site" means the supervisor ends the current session and starts a new one in the 3D site's world, with that site's `index.md` as the persona. Since your orchestrator already starts fresh sessions per ticket, this is the same shape as what you have.
- **Context crosses via files, not memory.** After the respawn the agent can't see the previous world, so handoff has to be an explicit artifact: a short summary plus whatever landed in the shared project. For the video-needs-3D case, the video session writes a request into the shared project, the orchestrator switches to the

3D site, which does the work and writes its output to the shared project, and then switching back picks up from there. That makes the handoff inspectable, and it works across a crash.

- **Switching authority stays outside the sandbox.** The agent calls a "switch site" tool over MCP, and the supervisor decides whether to honour it. If the agent could pick its own grants it could widen them, so the request goes through the supervisor, which only ever builds a world from a site's own manifest.
- **The default agent needs a small world of its own.** Probably the site list, ticket state, and handoff notes, but not the contents of any site's projects. It orchestrates without being able to touch everything.

One thing to decide is how much of the previous site's persona and context should survive a switch. Fully clean is the safest (no leaked assumptions, which was your concern with mixed harnesses). A short carried-over task summary is usually enough, and it's the only thing that should cross.

> yeah think of it like a minimal sub session, so tehnically there is only an orchestrator and you just jump to the site

That's a cleaner model than what I was describing, because there's only one long-lived thing (the orchestrator) and everything else is a scoped sub-session it opens. A site stops being a place an agent lives and becomes a function the orchestrator calls: enter with a task, get a bounded world, run, return a result.

That has a few nice consequences:

- **The sub-session is the sandbox unit.** Spawning one means building the namespace from the site's manifest, loading the site's `index.md` as the persona, mounting the task overlay, and running. When it ends, the namespace is torn down and only the overlay and a result remain. There's no state to leak, because the world is built fresh each time.
- **The orchestrator's own context is the continuity.** Since it stays alive across jumps, it can hold the conversation with you, decide when to jump, and receive a summary when a sub-session returns. Handoff between sites then goes through the orchestrator, so the 3D site never needs to know the video site exists.
- **Nesting becomes possible but should probably be capped.** If the video sub-session can itself "jump" to 3D, you get a stack, which brings back context and grant questions. Keeping it flat (a sub-session returns to the orchestrator, which decides the next jump) is simpler and keeps the authority in one place.
- **It matches how your tickets already work.** A ticket is a sub-session with an overlay and a reviewer, so a site jump is the same mechanism with a different manifest and persona.

The main design question is the return contract. A sub-session should come back with

a small structured result: status, a short summary, and pointers to what changed in the overlay or shared project. If that shape is fixed, the orchestrator can chain sites without understanding their internals, and your reviewer agent has a consistent thing to check.

for the current implementation tho, it normally finds stuff in like 2 s

Checking which search the user's timing refers to ▸

If you mean the orchestrator locating a site and starting a sub-session, the sandbox won't threaten that 2 s. Building a bubblewrap namespace with a few bind mounts and an overlay takes tens of milliseconds, and creating or discarding the overlay's `upper` directory is close to free. The time goes into loading the persona and getting the first model token, which is the same whether or not there's a sandbox.

The one thing that could add real cost is the manifest step. If building a slice means walking a big tree to build a symlink farm (thousands of media files, say), that scales with the tree. Bind-mounting whole directories or listed paths avoids it, since the cost then depends on the number of mounts and not the number of files.

I'm not sure whether "finds stuff" means entering a site or something like searching for files or assets, though. If it's search, a scoped world should help, because the agent has less to look through, and your brain embeddings server would just index per-grant instead of the whole disk. Which did you mean?

well you can already look up how ICM works, my ideas just expand on it

Searched the web ▸

That clears up the earlier "finds stuff" comment: you meant context lookup, not the sandbox. I hadn't connected it to ICM, Interpretable Context Methodology, which is a paper and set of workspaces on folder structure as agent architecture.

ICM replaces framework-level orchestration with filesystem structure: a folder hierarchy holds the prompts and context for each stage, so no coordination framework is needed. Stage order comes from folder numbering, context scoping from the hierarchy, and handoff between stages from one folder's output being the next folder's input. The agent reads downward and stops when it has enough, which is typically a few thousand tokens instead of tens of thousands. That fits your 2 s figure, since finding context is just reading a few files in the right folder. License: CC BY 4.0 +2

Your idea takes this further in a specific way. In ICM the folder is a context boundary that the agent respects, but nothing enforces it. An agent could still wander elsewhere on disk. You're proposing to make the folder the actual capability boundary, so what the agent can read and write is the folder and nothing else. Your additions on top of ICM:

- **Enforcement.** Namespace and mounts turn "the agent should only look here" into "the agent can only see here". This is what protects against mistakes.
- **Writable tasks without risk.** ICM workspaces are mostly sequential and self-contained. The overlay gives a site a safe way to work on shared projects, with the diff as the review artifact.
- **Cross-workspace sharing.** ICM treats a workspace as a copyable folder. Your sites sharing a project is a case it doesn't cover, and the grant model handles it.
- **The orchestrator as the only long-lived agent.** This is already close to ICM's one-orchestrating-agent idea. Your jump-into-a-site model is the same shape with a hard boundary around each jump.

Since ICM scopes context by layers (a rendering agent might only need the top few), your site's `index.md` could follow that layering convention. A sub-session would then load only the layers its task needs, and the manifest would decide which mounts exist.

Sources:

- Interpretable Context Methodology (arXiv)
- ICM repository
- ICM on enterprise level - introducing Taurus

> yeah the raeson why it doe not do the filesystem part is because its harness agnostic

That makes sense, and it's the real difference between the two. ICM has to work with whatever agent tool opens the folder (Claude Code, another CLI, a person), so it can only offer convention. It can't enforce anything, because it doesn't control how the agent process is launched.

Composer does control that. It owns the spawn, so it can turn ICM's folder conventions into actual boundaries at launch: the same folder structure, plus a namespace built from it. In practice that means a Composer site could simply be an ICM workspace, and the sandbox is an extra property Composer adds when it launches the sub-session. A workspace stays portable and still works unenforced in any other harness, which also keeps your layer optional.

It has one limit: your Claude Code external harness. Composer spawns the CLI, so it can wrap that process in the same namespace, but the CLI brings its own tools and its own ideas about where it can read and write. Inside the sandbox those just see a smaller world, which is fine. The things to check are that it doesn't need paths outside the namespace (its config and session directories, for example) and that your MCP socket is exposed into the namespace, as discussed earlier.

well i also added a frontend per site to it

Deciding where the frontend should run ▸

That solves the problem I flagged earlier about reviewing binary changes. A diff of a video or a mesh is meaningless, but a site's own frontend knows how to show its content: the video site can play the old and new clip side by side, and the 3D site can show a model viewer. The overlay's `upper` directory is the list of what changed, and the frontend decides how to render each entry. Merge or discard then becomes a button in a UI that already understands the content.

The frontend should probably live on the host side, outside the sandbox. It is the human-facing half, so it needs to read the site's folder, the shared projects, and the pending overlay, which the agent itself can't see in full. The agent and the frontend then talk through the same interface Composer already exposes over MCP, and neither needs direct access to the other's world.

The frontend's source also lives in the site folder, so there is a decision about who can edit it:

- **If the agent can edit its own frontend**, you get a site that adapts its UI as the workflow evolves, which is a strong fit with micro apps. The edits land in the overlay like any other change, so you review them before they go live.
- **If it can't**, the frontend is stable, but only you can change it.

Since your threat model is mistakes and not malice, the first is probably fine, as long as frontend changes pass through the same review as everything else. A broken frontend could otherwise hide the very diff you need to review it. A cheap safeguard is a plain fallback view (just the file list from `upper`) that always works, even when the site's custom UI is broken.

Does the frontend currently read files straight from the project directory, or does everything go through the agent?

like a 3D design workflow:

```
3D-designer/
  index.md
  01_idea/
  02_sketches/ <-- has svg files in them, with parameter files
  03_parts/
  ...
```

Mapping stage outputs to read-only inputs ▸

This layout makes the grants almost free, because the numbering already says who can

touch what. A sub-session working on stage N gets:

- `01_` through `0N-1_` read-only, since they are its inputs
- `0N_` writable, via the overlay, since that is its output
- nothing from later stages

So the manifest I kept talking about is implied by the folder order. Add a stage and the grants follow. It also keeps a stage honest: the sketch stage can't quietly edit the idea, and the parts stage can't rewrite the sketches it was given. When the parts stage finds a problem in a sketch, the fix is to jump back to `02_sketches/` as its own sub-session, which fits your orchestrator model.

SVG plus parameter files is also the best case for review. Both are text, so the diff is real: a changed parameter reads as one line in the overlay, and the frontend can render the old and new SVG side by side. Since the SVG presumably regenerates from the parameters, the parameter diff is the thing to approve, and the rendered image is just the confirmation.

Two things to settle in your ideation pass:

- **Staleness downstream.** If you merge a change to `02_sketches/`, everything in `03_parts/` built from the old sketches is now out of date. The filesystem can show this without extra machinery if each stage records a hash of its inputs, or if you just compare mtimes. The frontend can then mark `03_` as stale and offer to re-run it.
- **Binary outputs in later stages.** Parts and renders probably end up as meshes or images, where overlay copy-up and diffs get awkward. The reflink clone I mentioned earlier is the fallback there, with the frontend's viewer doing the comparison.

Does one sub-session cover a single stage, or can it run several stages in sequence? That decides whether the grants shift mid-session.

don't over engineer it

Fair. The minimal version is:

- One folder per site, numbered stages.
- When a sub-session starts, bind-mount earlier stages read-only and the current stage writable.
- Skip overlays, hashes, and staleness tracking for now.

Rollback can come from git or a plain snapshot of the folder before each run. The frontend just shows what changed. Everything else (overlays, reflinks, cgroups, the VM image) can wait until a real problem shows up.

> google's harness2 is also close to what i am building: google-research/harness2

Read github.com ▸

The README shows it overlaps with your project at the site level and diverges on what it's for.

What's close. Harness² treats a harness as a set of editable parts around a model: system prompt, guardrails, memory, tool guidance, subagents, skills, scripts, and plugins. That is nearly the contents of one of your site folders (`index.md` as the prompt, `workflows/` as skills, `scripts/` as scripts), and both approaches rest on the idea that the harness is the thing you tune per kind of work, not the model.

What's different. Harness² automates the tuning. For each task it proposes edits to the harness, probes them by running the task, and composes a refined harness from how the runs differ. A second level keeps lessons from each task to improve future refinement. Your sites are hand-written workflows with a person or the default agent switching between them, with the sandbox and frontend around them. It's a different bet: you write the workflow in detail so smaller models can follow it, while Harness² searches for better harnesses automatically.

Where they could meet. If you ever want a site to improve itself, Harness²'s propose-probe-compose loop maps onto your overlay idea. Candidate edits to a site's `workflows/` or `scripts/` land in an overlay, get tried on a real task, and you review the winning diff in the site's frontend before merging. That would be a later addition, though, not part of the minimal version.

One more difference: the README says executor network access is unrestricted and the Codex executor runs with its sandbox disabled. Its focus is benchmark performance, so it has none of the containment you're designing.

I only read the README, so I can't say how the harness components are stored or edited internally. The paper and docs would tell you if the on-disk format is worth borrowing.

Sources:

- google-research/harness2

> nah. i went with a simpler model: i just let the model write down its workflows and let it improve them that way. and that also works

That's the simpler approach, and it fits the layout we already landed on. `workflows/` and `scripts/` were the agent's own writable space, so "write down what worked and refine it next time" is just the agent editing files in its home. There's no separate

improvement machinery, because the site folder is the memory.

The only thing worth having is an undo for bad edits. A workflow that gets worse is quiet damage, since nothing errors, and the next session just follows the worse instructions. Keeping each site folder in git with a commit per session covers it, and `git diff` on `workflows/` shows exactly what the agent changed about its own behavior. That's the same minimal rollback you already planned, with no extra system.

> for me the "harness" should be the immutable space, and the tools are whats discovered when moving through sites

Rechecking whether agents still write workflows ▸

That's a clean split, and it simplifies the sandbox. The harness (core prompt, guardrails, how to use Composer over MCP) is one read-only mount, identical in every sub-session. The agent can't drift it, because it can't write there. Everything that changes lives in the site: `workflows/` and `scripts/` are the tools it finds on entry and extends as it works.

That also settles the rollback worry from before. A bad self-edit can only damage a site's tools, never the harness, so the worst case stays local to one site and `git diff` on that folder shows it. And the harness stays small and stable, which suits your smaller models.

It also changes what `index.md` is. It becomes the site's tool catalog (what's here, when to use it) instead of a persona. When the orchestrator jumps in, it reads that file and knows what it has to work with, which is the discovery step you described. The one thing to decide is where the "personality" you mentioned lives. If it's just a short note in the site's `index.md` on top of the fixed harness, nothing about the immutability changes.

> the problem with ICM is that it doesn't work with coding, but my way does

Comparing ICM's stage-sequential folders with coding project layouts ▸

That matches what I read, with one caveat. ICM is built around numbered stages where one folder's output feeds the next, and the paper lists concurrent execution as sequential by design. That suits pipelines like reporting or content production. Code doesn't fit it, because a repository has its own layout (`src/` , `tests/` , build files) that tooling expects, and the work loops (write, test, fix, review) instead of moving forward through stages. You can't reorganize a repo into numbered folders without breaking it. People do launch coding agents inside ICM workspaces, but there the workspace holds the process around the code, not the code itself.

Your split avoids this because the project and the site are separate things. The code

lives in the shared project in its native layout, mounted writable for the coder site. The site holds the workflow (how this kind of work gets done, the scripts, the reviewer loop) and imposes no structure on the repo. The same site can then work on any repo, and the repo doesn't need to know it's being worked on.

The 3D designer layout is the case where numbered stages do fit, since the work really is a pipeline. So the two aren't exclusive: stages are one way a site can organize its own folder when the work is sequential, and a coding site just doesn't use them. The mechanism is the same either way (a fixed harness, a site folder of tools, and projects mounted in).

